Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 4

По дисциплине: «Современные методы защиты компьютерных систем» Тема: «Сбор, нормализация и корреляция событий информационной безопасности в системах централизованного мониторинга»

Выполнил:
Студент 4 курса
Группы ИИ-26
Згера Е. А.
Проверила:
Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение развертывания SIEM на базе Wazuh 4.7.3, настройки агента сбора журналов (веб-сервер Nginx и системный журнал Linux) и написания пользовательских правил корреляции для выявления веб-атак.

Вариант 14

14	Подделка запросов (SSRF): `169.254.169.254`	`169.254.169.254`	100014	12	T1552.005 (Cloud Instance Metadata API)

Ход работы

Развертывание стенда:

```
└─$ docker compose ps
WARN[0000] /home/kosem/wazuh-lab/wazuh-docker/single-node/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove
it to avoid potential confusion
NAME                             IMAGE                          COMMAND                  SERVICE           CREATED         STATUS      PORTS
single-node-target-agent-1       single-node-target-agent       "/start.sh"              target-agent      5 minutes ago   Up 5 minutes
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
single-node-wazuh.dashboard-1    wazuh/wazuh-dashboard:4.7.3    "/entrypoint.sh"         wazuh.dashboard   5 minutes ago   Up 5 minutes
443/tcp, 0.0.0.0:443->5601/tcp, [::]:443->5601/tcp
single-node-wazuh.indexer-1      wazuh/wazuh-indexer:4.7.3      "/entrypoint.sh open "   wazuh.indexer     5 minutes ago   Up 5 minutes…
0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp
single-node-wazuh.manager-1      wazuh/wazuh-manager:4.7.3      "/init"                  wazuh.manager     5 minutes ago   Up 5 minutes
0.0.0.0:1514-1515->1514-1515/tcp, [::]:1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp, 0.0.0.0:55000->55000/tcp, [::]:55000-
>55000/tcp, 1516/tcp
```

Проверка контрольных точек:

1. Индексер отвечает и принимает пароль. Порт 9200 публикуется на хост: в docker compose ps видно 0.0.0.0:9200->9200/tcp. Проверка в Ubuntu:

```
└─$ curl -k -u admin:SecretPassword https://127.0.0.1:9200
{
  "name" : "wazuh.indexer",
  "cluster_name" : "opensearch",
  "cluster_uuid" : "a8v5sj2MQo-1XAZW4dhf8Q",
  "version" : {
    "number" : "7.10.2",
    "build_type" : "rpm",
    "build_hash" : "db90a415ff2fd428b4f7b3f800a51dc229287cb4",
    "build_date" : "2023-06-03T06:24:25.112415503Z",
    "build_snapshot" : false,
    "lucene_version" : "9.6.0",
    "minimum_wire_compatibility_version" : "7.10.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "The OpenSearch Project: https://opensearch.org/"
}
```

2. Менеджер подключен к индексеру:

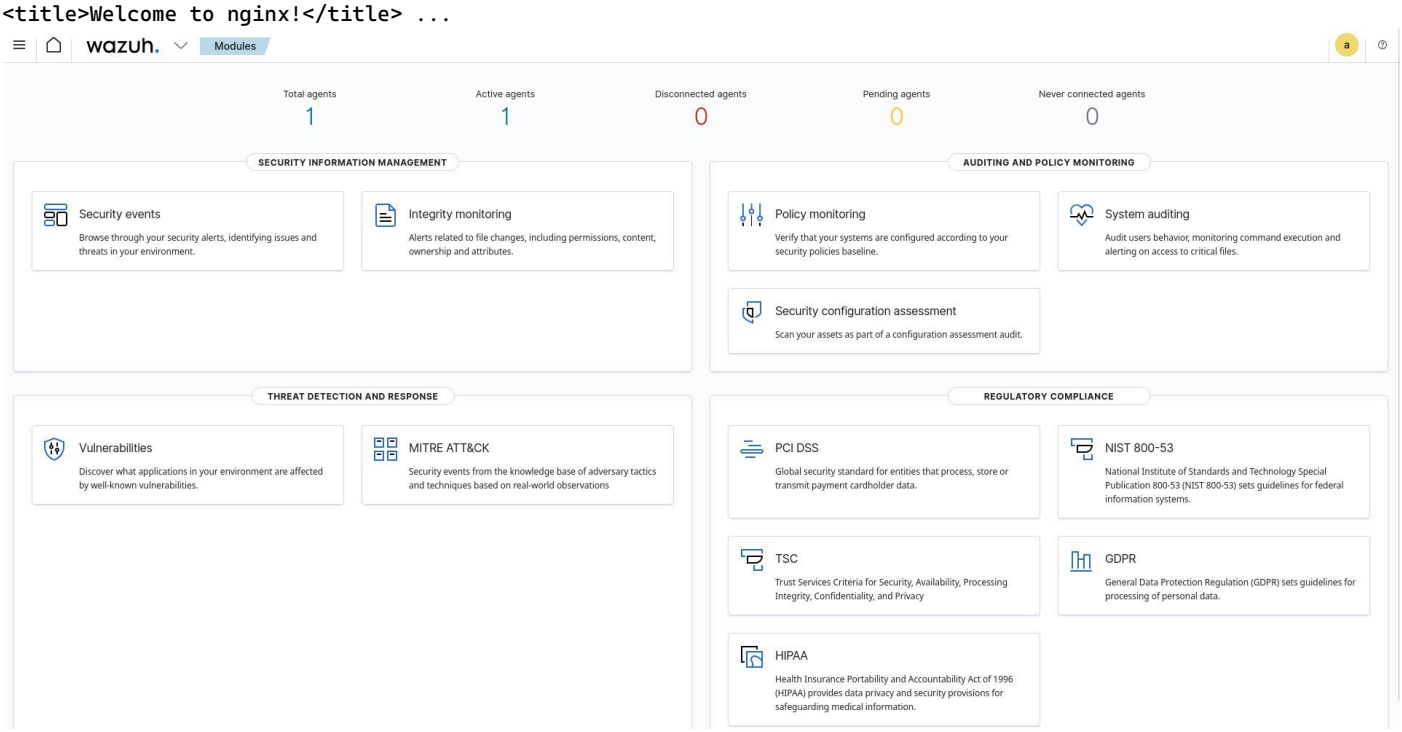
```
└─$ docker compose exec wazuh.manager filebeat test output
elasticsearch: https://wazuh.indexer:9200...   parse url... OK
connection...
    parse host... OK     dns
lookup... OK     addresses:
172.19.0.2     dial up...
OK
  TLS...     security: server's certificate chain verification is
enabled     handshake... OK     TLS version: TLSv1.3     dial up... OK
talk to server... OK   version: 7.10.2
```

3. Агент видит менеджер по имени:

```
└─$ docker compose exec target-agent getent hosts wazuh.manager
172.19.0.3      wazuh.manager
```

4. Веб-интерфейс и целевой сервер. Откройте в браузере https://127.0.0.1, примите предупреждение о самоподписанном сертификате и войдите как admin / SecretPassword. Затем выполните curl http://127.0.0.1:8080: должна вернуться страница с текстом «Welcome to nginx».

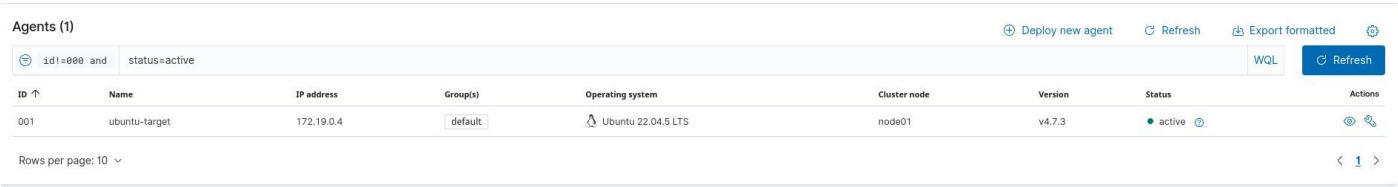
```
└─$ curl http://127.0.0.1:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title> ...
```



Задания

Задание 1. Подключение агента и маршрутизация логов

1. В Dashboard откройте Wazuh → Agents. Агент с именем ubuntu-target должен быть в статусе Active.



2. Проверьте, какие файлы уже мониторятся: docker compose exec target-agent grep -n "<location>" /var/ossec/etc/ossec.conf

```
└─$ docker compose exec target-agent grep -n "<location>" /var/ossec/etc/ossec.conf
181:    <location>/var/log/nginx/access.log</location>
186:    <location>/var/log/nginx/error.log</location>
191:    <location>/var/ossec/logs/active-responses.log</location> 196:
<location>/var/log/dpkg.log</location>
```

Откройте конфигурацию агента:

```
docker compose exec target-agent nano /var/ossec/etc/ossec.conf
```

Внутри элемента <ossec_config> добавьте блок <localfile> для nginx:

```
<localfile>
    <log_format>syslog</log_format>
    <location>/var/log/nginx/access.log</location>
  </localfile>
  <localfile>
```

```
    <log_format>syslog</log_format>
    <location>/var/log/auth.log</location>
  </localfile>
```

Блок для /var/log/auth.log добавьте только в том случае, если в выводе grep его нет: в стандартной конфигурации Ubuntu он, как правило, уже есть, и дубликат не нужен. Сохраните файл (Ctrl+O, затем Enter) и закройте nano (Ctrl+X).

3. Перезапустите агента:

```
docker compose exec target-agent /var/ossec/bin/wazuh-control restart
```

4. Сгенерируйте события:

- выполните 5 запросов curl http://127.0.0.1:8080;

```
└─$ for i in {1..5}; do
    curl -s http://127.0.0.1:8080 > /dev/null done
```

- запишите тестовое сообщение в системный журнал:

```
docker compose exec target-agent logger -p auth.notice "lab4 test auth event"
```

```
└─$ docker compose exec target-agent tail -n 5 /var/log/nginx/access.log
172.19.0.1 - - [02/Nov/2026:15:46:27 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/8.22.0" 172.19.0.1 - -

[02/Nov/2026:15:46:27 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/8.22.0" 172.19.0.1 - -

[02/Nov/2026:15:46:27 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/8.22.0" 172.19.0.1 - -

[02/Nov/2026:15:46:27 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/8.22.0"

172.19.0.1 - - [02/Nov/2026:15:46:27 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/8.22.0"
└─$ docker compose exec target-agent grep "lab4 test auth event" /var/log/auth.log
Nov  2 17:52:47 ubuntu-target sshd[1234]: lab4 test auth event
```

5. В Dashboard откройте Security events и отфильтруйте события по агенту ubuntu-target. Должны быть видны события nginx, разобранные декодерами на поля (IP-адрес источника, URL, код ответа), и тестовое сообщение из auth.log. События появляются через 10–30 секунд.



Задание 2. Написание пользовательских правил корреляции

1. В Dashboard откройте Management → Rules → Manage rule files и выберите файл local_rules.xml. Названия пунктов меню могут немного отличаться в зависимости от версии интерфейса.
2. Каждое правило обязательно должно находиться внутри элемента <group>. Шаблон структуры:

```
<group name="local,web,attack,">
  <rule id="ID_ИЗ_ВАРИАНТА" level="УРОВЕНЬ_ИЗ_ВАРИАНТА">
    <if_sid>31100</if_sid>
    <match>СИГНАТУРА_ИЗ_ВАРИАНТА</match>
    <description>Краткое описание атаки</description>
    <mitre>
      <id>ТЕХНИКА_MITRE_ИЗ_ВАРИАНТА</id>
    </mitre>
  </rule>
</group>
```

Подставьте значения из вашего варианта. Сигнатуру берите из таблицы вариантов в закодированном виде (см. задание 3).

3. Если в файле уже есть пример правила с таким же ID, замените его. Повторяющийся ID не даст менеджеру загрузить правила.

local_rules.xml

```
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

</group>

<group name="local,web,attack,">
  <rule id="100014" level="12">
    <if_sid>31100</if_sid>
    <match>169.254.169.254</match>
    <description>SSRF attack attempt: Cloud Instance Metadata API</description>
    <mitre>
      <id>T1552.005</id>
    </mitre>
  </rule>
</group>
```

4. Сохраните файл и перезапустите менеджер:

```
docker compose restart wazuh.manager docker compose exec wazuh.manager tail -n
50 /var/ossec/logs/ossec.log
```

В журнале не должно быть ошибок, связанных с правилами.

5. Проверьте правило через wazuh-logtest (обязательно для каждого варианта). Отправьте запрос своего варианта из задания 3 (пункт 2), затем скопируйте последнюю строку журнала агента:

```
docker compose exec target-agent tail -n 1 /var/log/nginx/access.log
```

и вставьте ее в утилиту проверки правил:

```
docker compose exec wazuh.manager /var/ossec/bin/wazuh-logtest
```

В выводе должна появиться фаза Completed filtering (rules) с ID вашего правила. Если правила нет, сначала проверьте, что строка из access.log содержит сигнатуру дословно.
Тестовый SSRF запрос:

```
└─$ curl -g --path-as-is 'http://127.0.0.1:8080/?url=http://169.254.169.254/latest/meta-data/'
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>     body {
width: 35em;
margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and working. Further
configuration is required.</p>
<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Последняя строка журнала nginx:

```
└─$ docker compose exec target-agent tail -n 1 /var/log/nginx/access.log
172.19.0.1 - - [02/Nov/2026:18:21:04 +0000] "GET /?url=http://169.254.169.254/latest/meta-data/
HTTP/1.1" 200 612 "-" "curl/8.22.0"
```

Проверка срабатывания правила:

```
└─$ docker compose exec wazuh.manager /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.7.3
Type one log per line
172.19.0.1 - - [02/Nov/2026:18:21:04 +0000] "GET /?url=http://169.254.169.254/latest/meta-data/
HTTP/1.1" 200 612 "-" "curl/8.22.0"
**Phase 1: Completed pre-decoding.
        full event: '172.19.0.1 - - [02/Nov/2026:18:21:04 +0000] "GET
/?url=http://169.254.169.254/latest/meta-data/ HTTP/1.1" 200 612 "-" "curl/8.22.0"'
**Phase 2: Completed decoding.
        name: 'web-accesslog'
        id: '200' protocol:
        'GET' srcip:
        '172.19.0.1'
        url: '/?url=http://169.254.169.254/latest/meta-data/'
**Phase 3: Completed filtering (rules).
        id: '100014' level:
        '12'
        description: 'SSRF attack attempt: Cloud Instance Metadata API'
        groups: '['local', 'web', 'attack']' firedtimes: '1' mail: 'True'
        mitre.id: '['T1552.005']' mitre.tactic:
        '['Credential Access']'
        mitre.technique: '['Cloud Instance Metadata API']' **Alert to
be generated.
```

Задание 3. Симуляция атак и аудит инцидентов

1. Сигнатуры в таблице вариантов записаны в том виде, в котором они попадают в access.log.

Поэтому спецсимволы в запросе нужно передавать в закодированном виде:

Символ	Как передать в URL
пробел	%20
< / >	%3C / %3E
"	%22
;	%3B
{ / }	%7B / %7D

2. Используйте curl с двумя флагами:

- --path-as-is: curl по умолчанию удаляет из пути последовательности ../, и без этого флага атака обхода каталога не дойдет до сервера в исходном виде;
- -g: отключает разбор фигурных скобок {} как шаблонов curl.

Шаблон команды: curl -g --path-as-is 'http://127.0.0.1:8080<ПУТЬ>'.
Используйте одинарные кавычки, а не двойные: в интерактивном bash символ ! внутри двойных кавычек запускает history expansion, и запрос варианта 7 не отправится.
Подставьте в <ПУТЬ> запрос своего варианта. Сигнатура из таблицы вариантов должна входить в строку запроса дословно, в закодированном виде:

14	Подделка запросов (SSRF): 169.254.169.254	169.254.169.254	100014	12	T1552.005 (Cloud Instance Metadata API)

Пример можно заменить любым другим запросом, если сигнатура остается в нем без изменений.

```
└─$ curl -g --path-as-is 'http://127.0.0.1:8080/?url=http://169.254.169.254/latest/meta-data/'

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title> ...
```

3. Убедитесь, что строка попала в журнал:

```
docker compose exec target-agent tail -n 1 /var/log/nginx/access.log
```

Сигнатура из вашего варианта должна быть видна в строке в том же виде, что и в правиле.

```
└─$ docker compose exec target-agent tail -n 1 /var/log/nginx/access.log
172.19.0.1 - - [02/Nov/2026:21:13:15 +0000] "GET /?url=http://169.254.169.254/latest/meta-data/ HTTP/1.1"
200 612 "-" "curl/8.22.0"
```

4. Откройте Security events в Dashboard. Алерт появляется через 10–30 секунд. Встроенные правила Wazuh могут сработать на ту же строку. В отчет берите именно алерт с ID вашего правила.

Security Alerts

Time ↓	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
Nov 3, 2026 @ 00:13:17.456	001	ubuntu-target	T1552.005	Credential Access	SSRF attack attempt: Cloud Instance Metadata API	12	100014

5. Найдите алерт и проверьте: инцидент имеет заданный Level, правильно определен ID правила, в деталях виден сработавший match, корректно отображаются метаданные атаки (IP-адрес злоумышленника, целевой URI, ID техники MITRE ATT&CK). IP-адрес в алерте будет адресом шлюза Docker-сети (обычно 172.x.0.1), а не 127.0.0.1: запрос идет через проброс портов с хоста. Это нормально, в отчете укажите именно этот адрес.

@timestamp	2026-11-02T21:13:17.456Z
_id	cokcnqEBQ0bPITr6J9am
agent.id	001
agent.ip	172.19.0.4
agent.name	ubuntu-target
data.id	200
data.protocol	GET
data.srcip	172.19.0.1
data.url	/?url=http://169.254.169.254/latest/meta-data/
decoder.name	web-accesslog
full_log	172.19.0.1 - - [02/Nov/2026:21:13:15 +0000] "GET /?url=http://169.254.169.254/latest/meta-data/ HTTP/1.1" 200 612 "-" "curl/8.22.0"
id	1793653997.1487506
input.type	log
location	/var/log/nginx/access.log
manager.name	wazuh.manager
rule.description	SSRF attack attempt: Cloud Instance Metadata API
rule.firedtimes	1
rule.groups	local, web, attack
rule.id	100014
rule.level	12
rule.mail	true
rule.mitre.id	T1552.005
rule.mitre.tactic	Credential Access
rule.mitre.technique	Cloud Instance Metadata API
timestamp	2026-11-02T21:13:17.456+0000

Вывод: в ходе работы была установлена и настроена SIEM-система Wazuh, организован сбор логов и реализовано обнаружение SSRF-атаки с помощью пользовательского правила. Были освоены базовые навыки анализа событий и выявления угроз.

SIEM является важной частью SOC, поскольку обеспечивает централизованный мониторинг инфраструктуры, обнаружение подозрительной активности и помогает специалистам оперативно реагировать на инциденты.